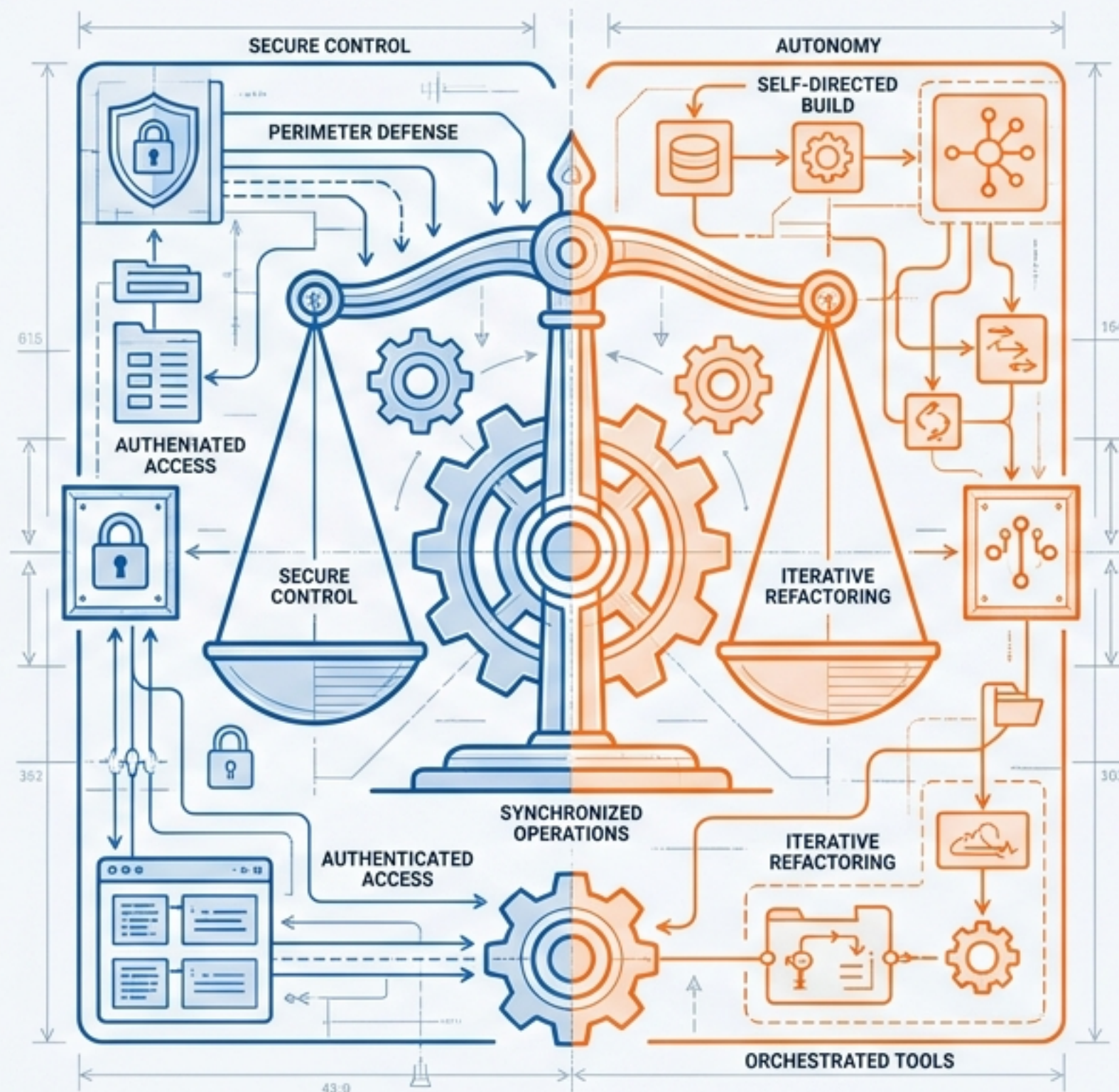


통제된 환경에서 완벽한 자율성으로 엔터프라이즈 AI 코딩 에이전트 구축

DX·AX 전략 기획 심화 실증 - 18일차 오후 세션

핵심 학습 목표

- Ollama 기반 프라이빗 로컬 코딩 환경 구축
- 터미널 네이티브 에이전트의 권한 통제 및 자율 빌드
- 1M 컨텍스트를 활용한 대규모 레거시 리팩토링
- 실무 상황별 툴스케이프(Toolscape) 오케스트레이션 전략



기업 보안과 클라우드 AI 코딩 도입의 치명적 딜레마

THE RISK

클라우드 LLM의 한계

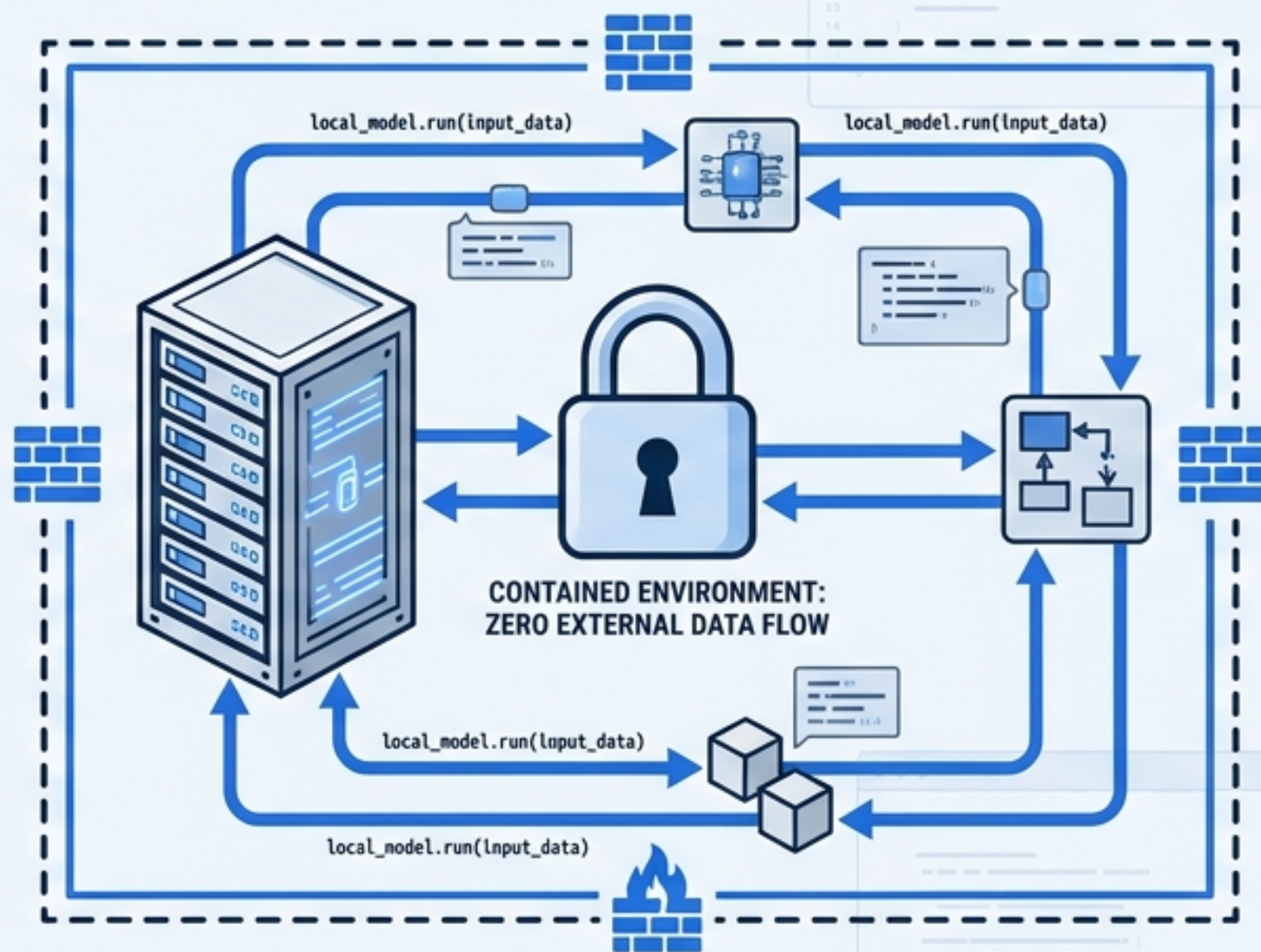
망분리 환경 및 소스코드 유출 리스크로 인한
엔터프라이즈 도입 불가



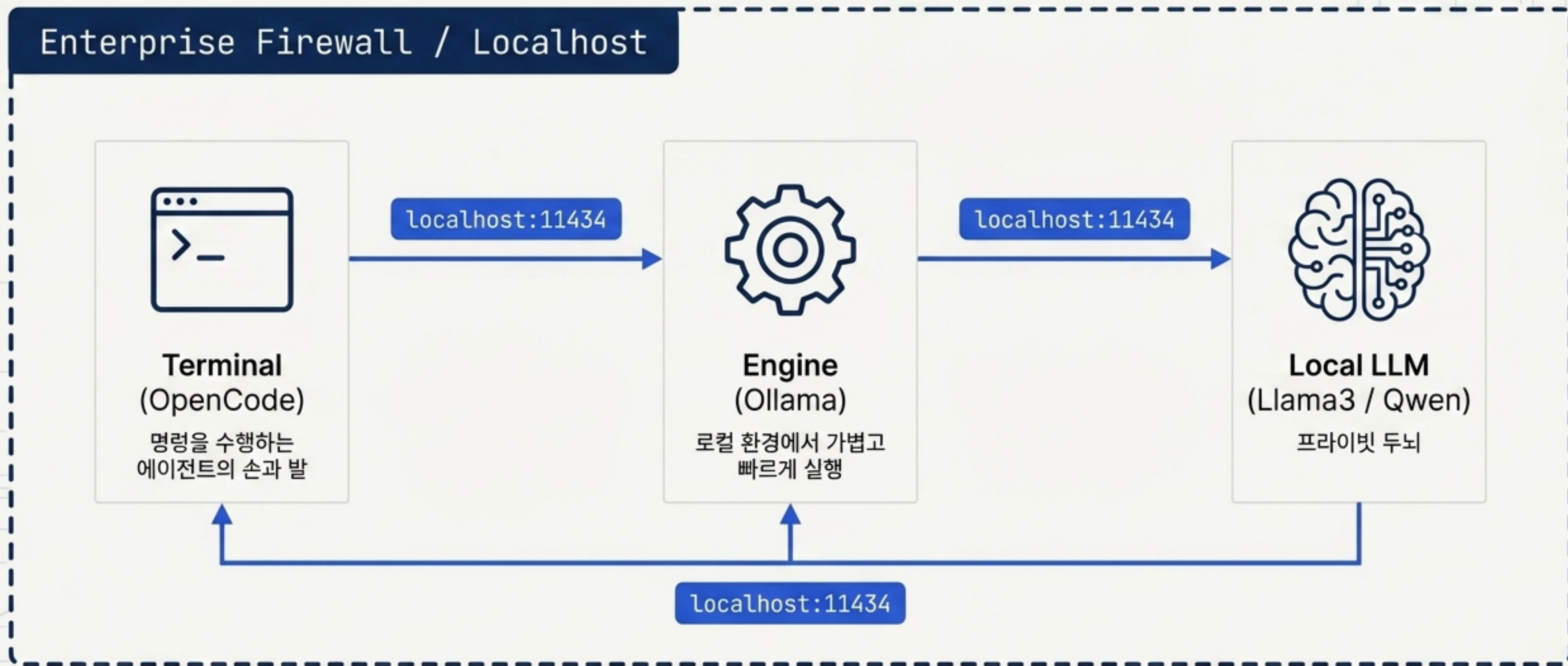
THE SOLUTION

프라이빗 샌드박스

Ollama 기반의 완전한 로컬(On-Premise)
구동으로 자산 보호

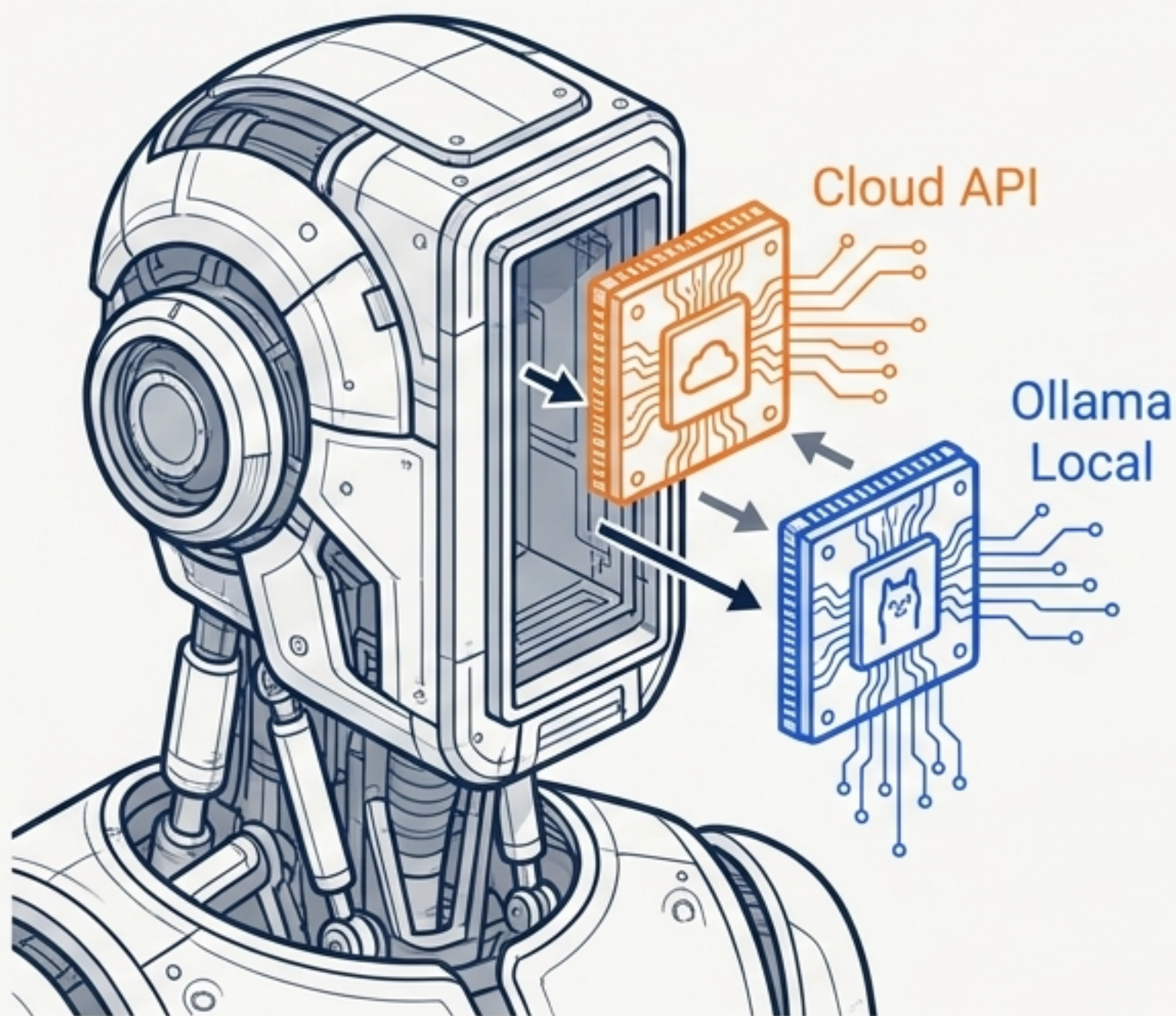


완벽한 오프라인 샌드박스: 로컬 에이전트 연동 아키텍처



Provider 핫스왑: 클라우드 두뇌에서 프라이빗 네트워크로

[실습 목표] Wi-Fi 차단 상태에서 단독 코드 리팩토링 수행 검증



Step 1: 엔진 설치 및 모델 다운로드

```
Terminal × +  
$ ollama run llama3
```

Step 2: OpenCode 설정 진입

```
Terminal × +  
$ opencode config
```

Step 3: Provider 변경 및 포트 바인딩

```
Terminal × +  
Provider: Ollama  
Endpoint: http://localhost:11434
```

Hands-on: 프라이빗 코딩 환경 구축 실습

네트워크가 없는 오프라인 상태에서 에이전트의 응답 속도 및 품질 확인



Terminal × +

- ☐ [실습 미션 1] 호스트 PC의 Wi-Fi 및 외부 네트워크 완벽 차단
- ☐ [실습 미션 2] 터미널에서 로컬 모델 정상 로드 확인
- ☐ [실습 미션 3] OpenCode에 기존 코드 리팩토링 지시 및 생성 완료 확인



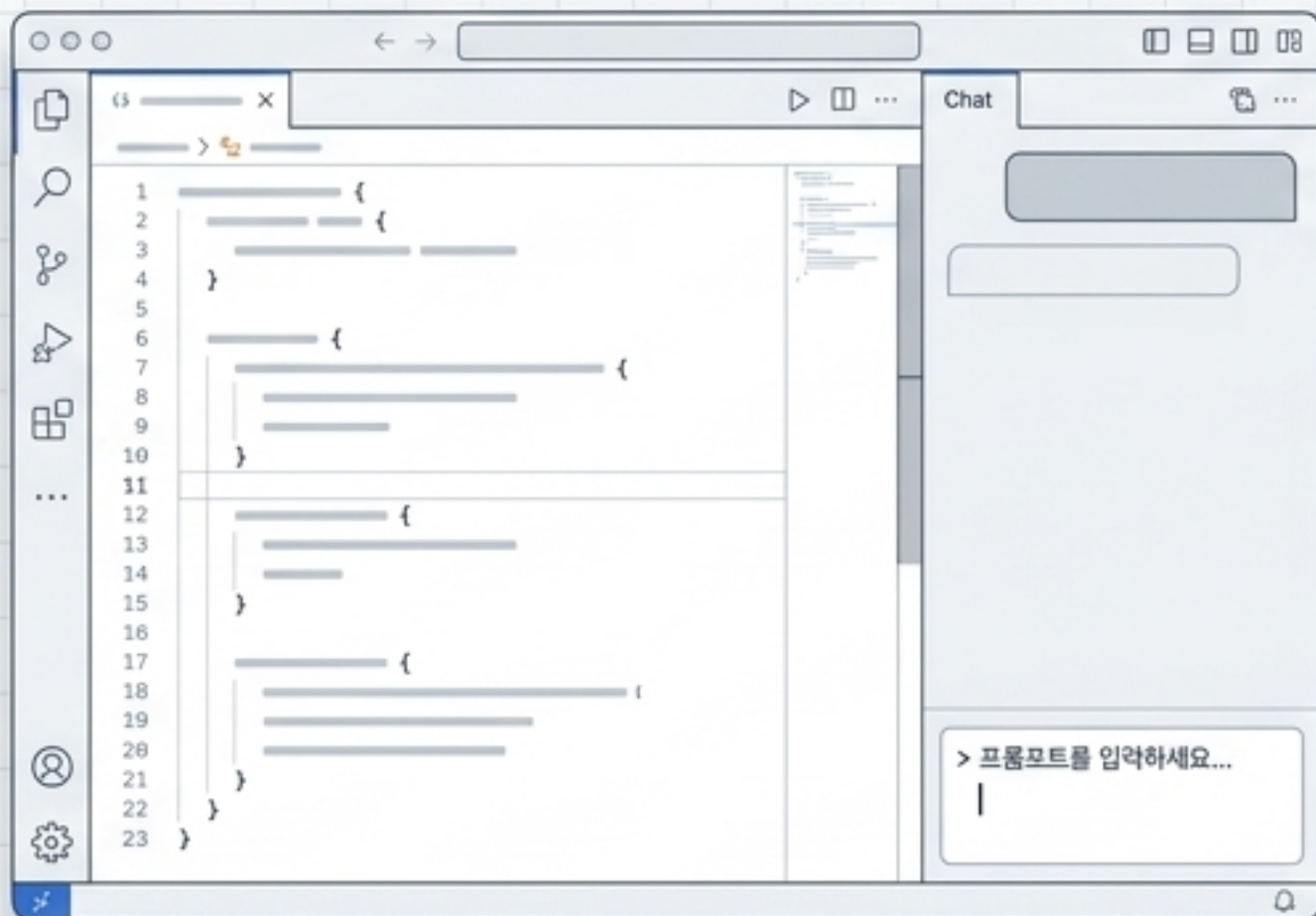
Pro-Tip: 로컬 모델 최적화 팁: 컨텍스트 윈도우의 한계를 고려하여 프롬프트를 최대한 간결하고 명확하게 작성하십시오.

패러다임 시프트: 수동적 코드 추천에서 능동적 터미널 에이전트로

에이전트의 진화: 패키지 설치, 파일 시스템 조작, 빌드, 테스트 자동화 권한 확보

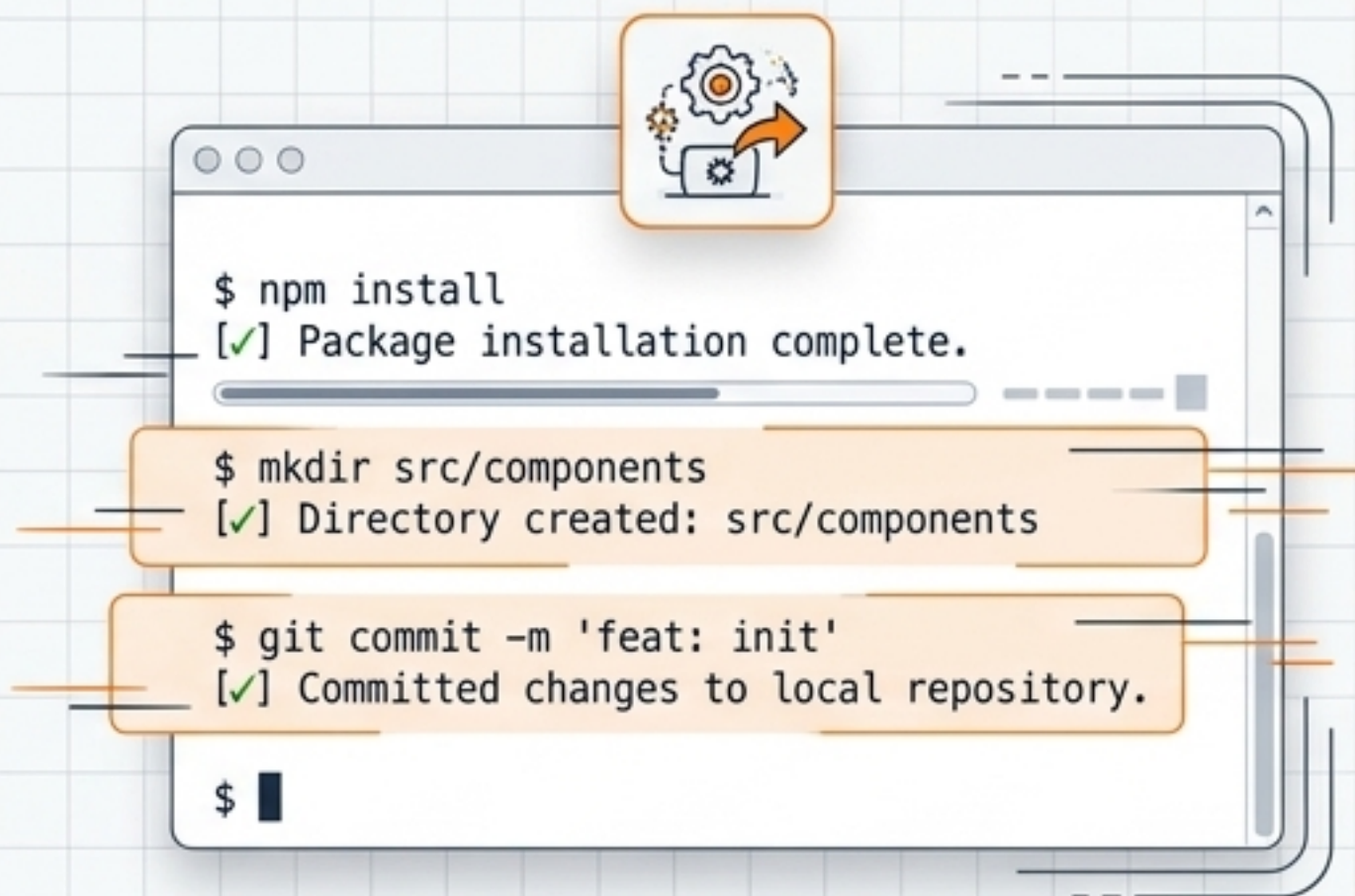
IDE 내장 AI (Passive)

사용자가 질문할 때만 응답하는 제한적 보조자

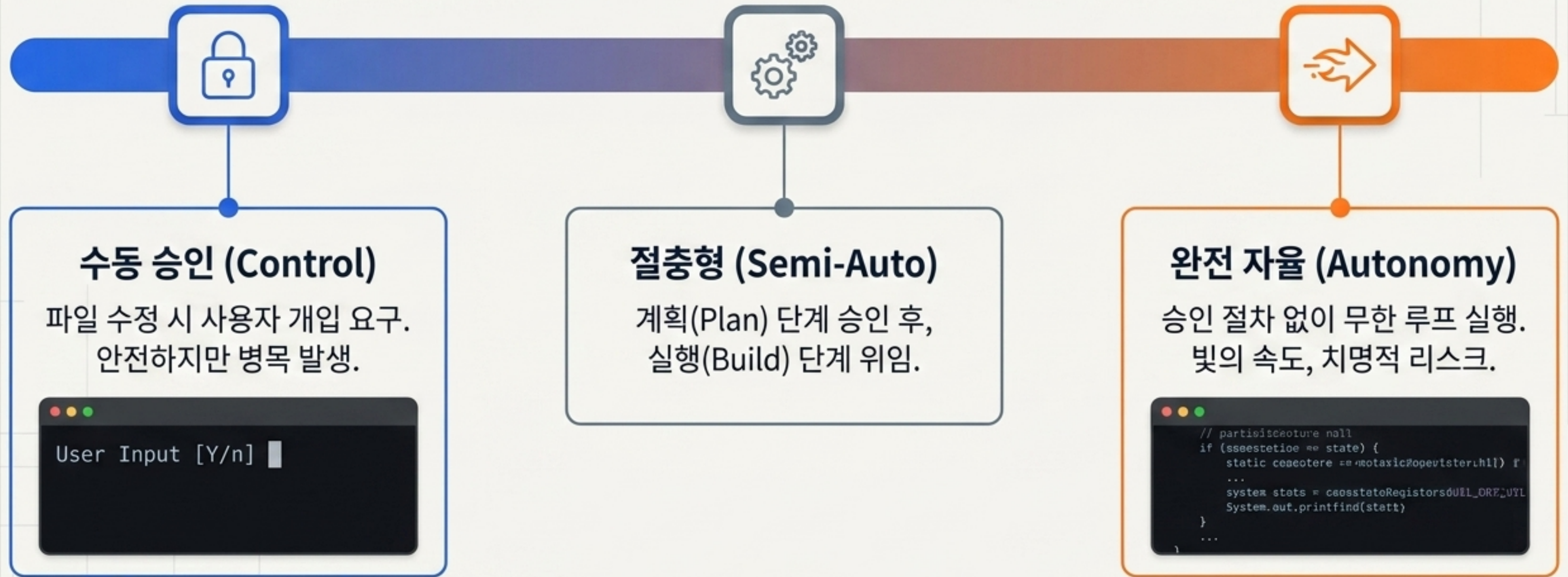


터미널 에이전트 (Active)

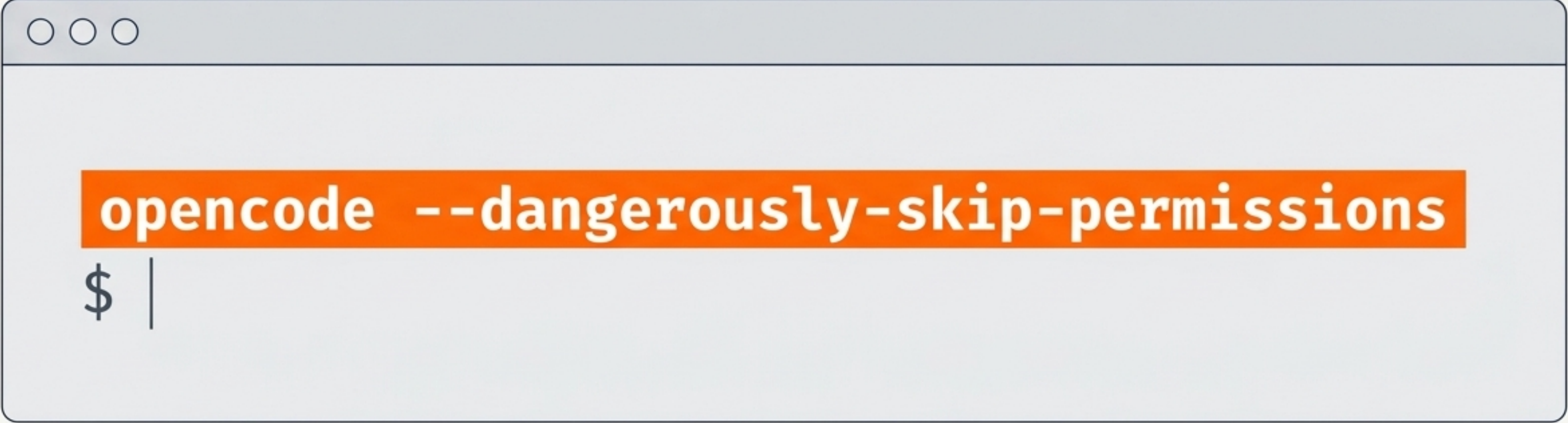
스스로 계획을 수립하고 OS 명령어를 실행하는 자율적 워커



권한 통제 스펙트럼: 개발 속도와 시스템 안전성의 트레이드오프



브레이크 없는 질주: 터미널 네이티브 YOLO 모드의 파괴력

A stylized illustration of a terminal window with a light blue background and a grey title bar containing three window control buttons. The terminal content shows a command being entered.

```
opencode --dangerously-skip-permissions  
$ |
```

YOLO (You Only Look Once) 모드



모든 보안 승인 프롬프트를 강제 우회하여 중간 개입 제로화.
명령어 한 줄로 패키지 설치, 파일 생성, 테스트까지 한 번에 완료.

Critical Risk (치명적 경고)

잘못된 프롬프트 주입 시 무한 에러 루프 및 시스템 파일 삭제
가능성 존재. 반드시 통제된 샌드박스에서 실행 요망.

자율 빌드를 통제하는 3단계 필수 안전장치 (Guardrails)



1. 복구 지점 생성
(Git Initialization)

작업 전 반드시 시스템 롤백 확보

```
git commit -m 'pre-agent'
```

2. 작업 환경 제한
(Scoped Directory)

에이전트가 접근할 수 있는
최상위 Workspace 폴더
강제 지정 및 격리

3. 단일 명확한 목표
(Single Clear Goal)

추상적 지시가 아닌,
구체적이고 닫힌 형태의
프롬프트 주입

Hands-on: 자율 프로젝트 빌드 런(Run)

[실습 미션] 신규 React 컴포넌트 5개를 포함한 대시보드 페이지 자율 생성

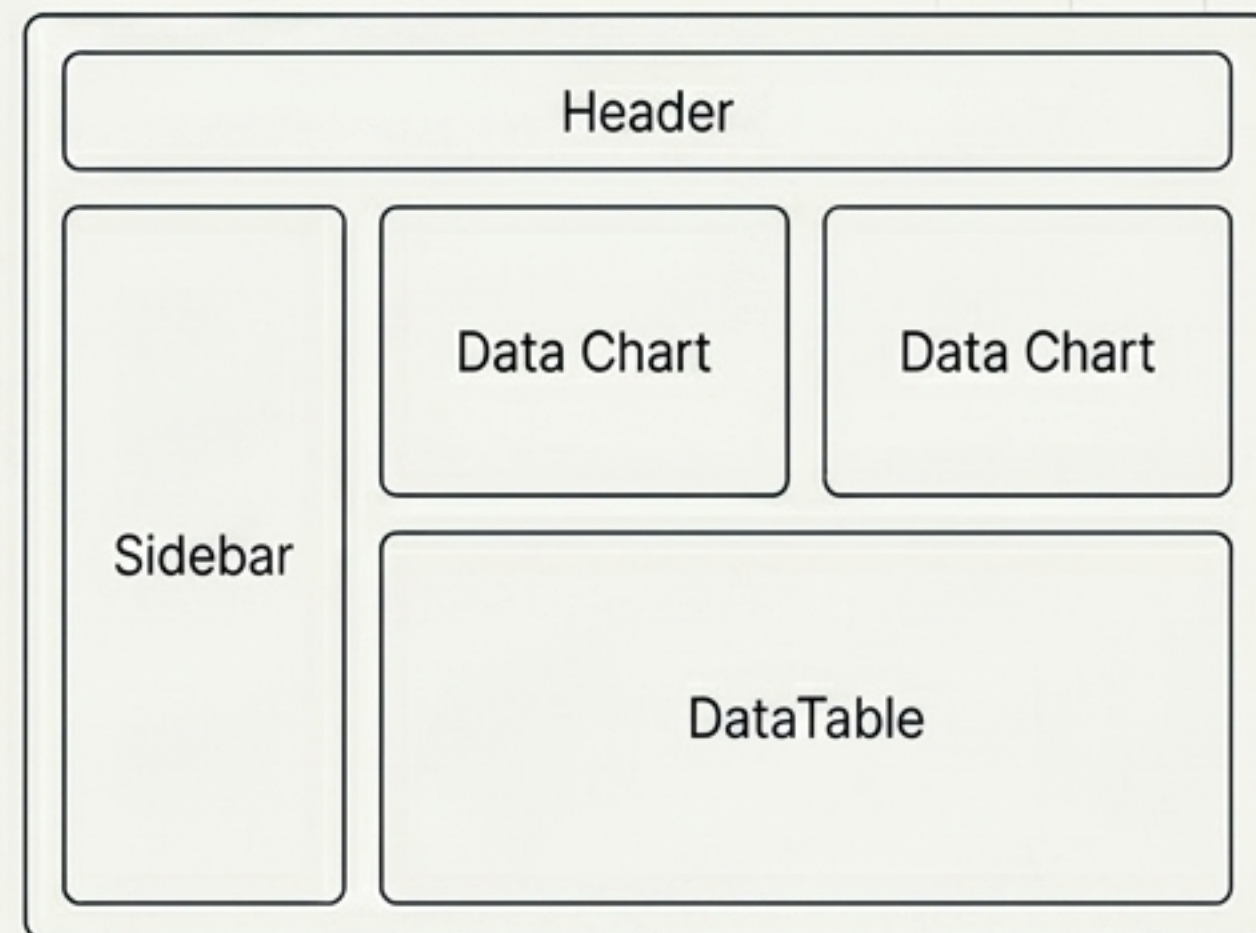
○○○ YOLO Flag Active

```
import React, { useState } from 'react';

const Dashboard = () => {
  const [data] = useState(dashboardData);
  return (
    <div className="dashboard">
      <Header user={data.user} />
      <Sidebar navigation={data.nav} />
      <div className="content">
        <DataChart type="sales" />
        <DataChart type="metrics" />
        <DataTable items={data.items} />
      </div>
    </div>
  );
};
```

중간 개입 없이
한 번에 빌드

Resulting Dashboard



핵심 관찰 포인트: 에러 발생 시 에이전트가 스스로 로그를 검색하고 디버깅하여 목표를 달성하는 사고 흐름(Reasoning) 파악.

1,000,000 토큰이 가져온 아키텍처 파악의 혁명

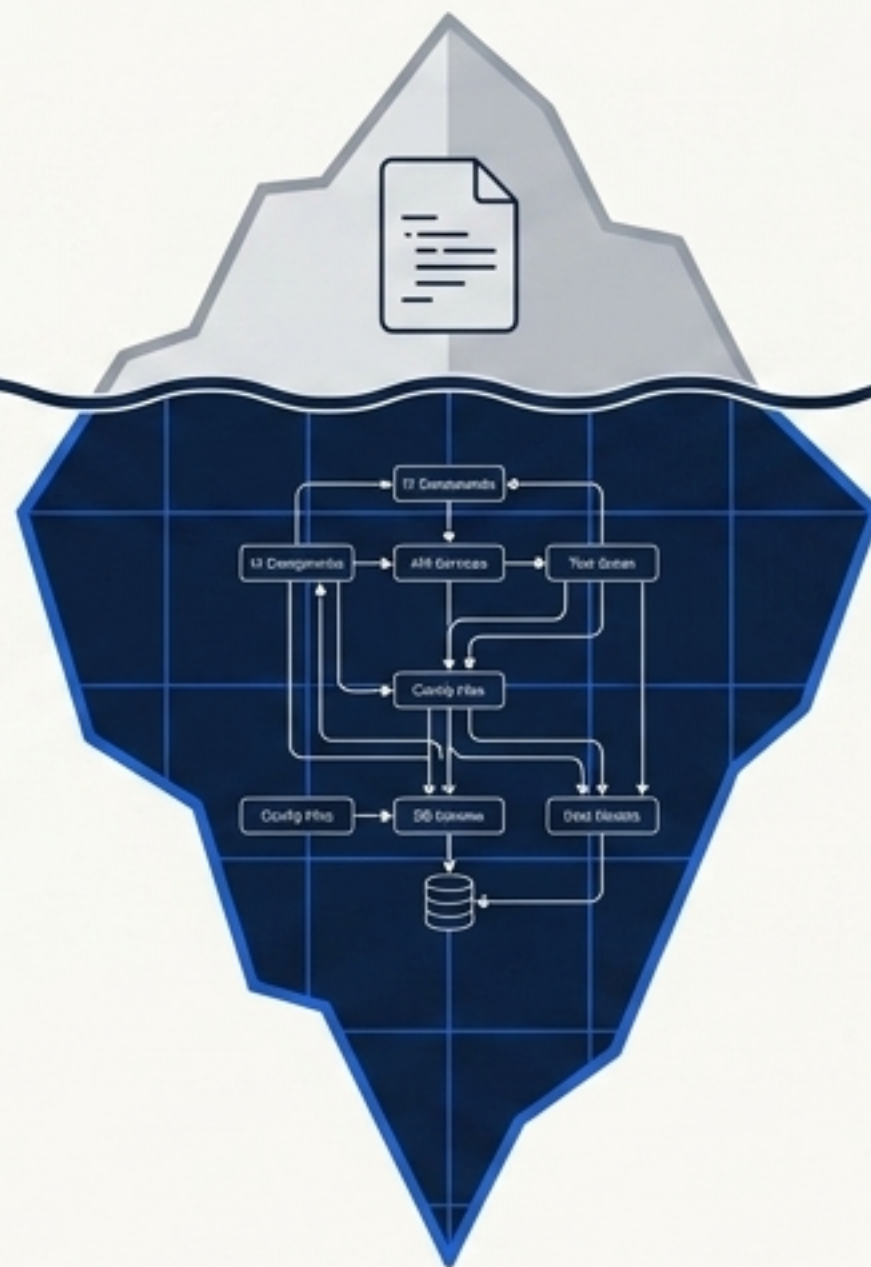
단순한 '용량' 증가가 아닌 AI의 인지적 '시야' 확장

기존 8K/32K 컨텍스트

단일 함수/파일 국소적 이해

1M (1,000,000) 컨텍스트

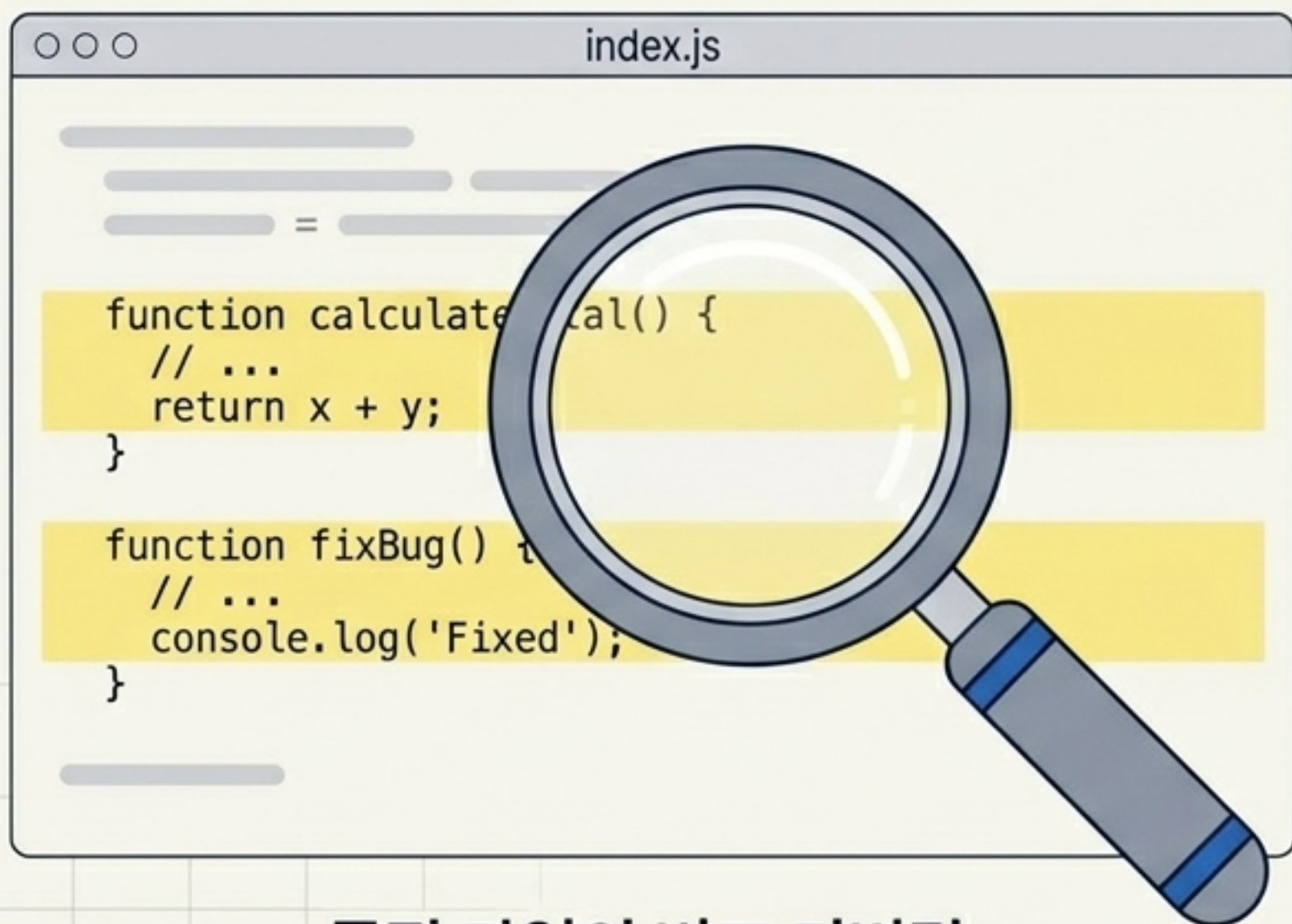
코드 11만 줄, 파일 100개, DB 스키마 등
프로젝트 생태계 전체 동시 파악



연쇄적인 사이드 이펙트를 사전에 예측하여 무결성 유지 (A 파일 수정 시 B, C 파일 자동 동기화)

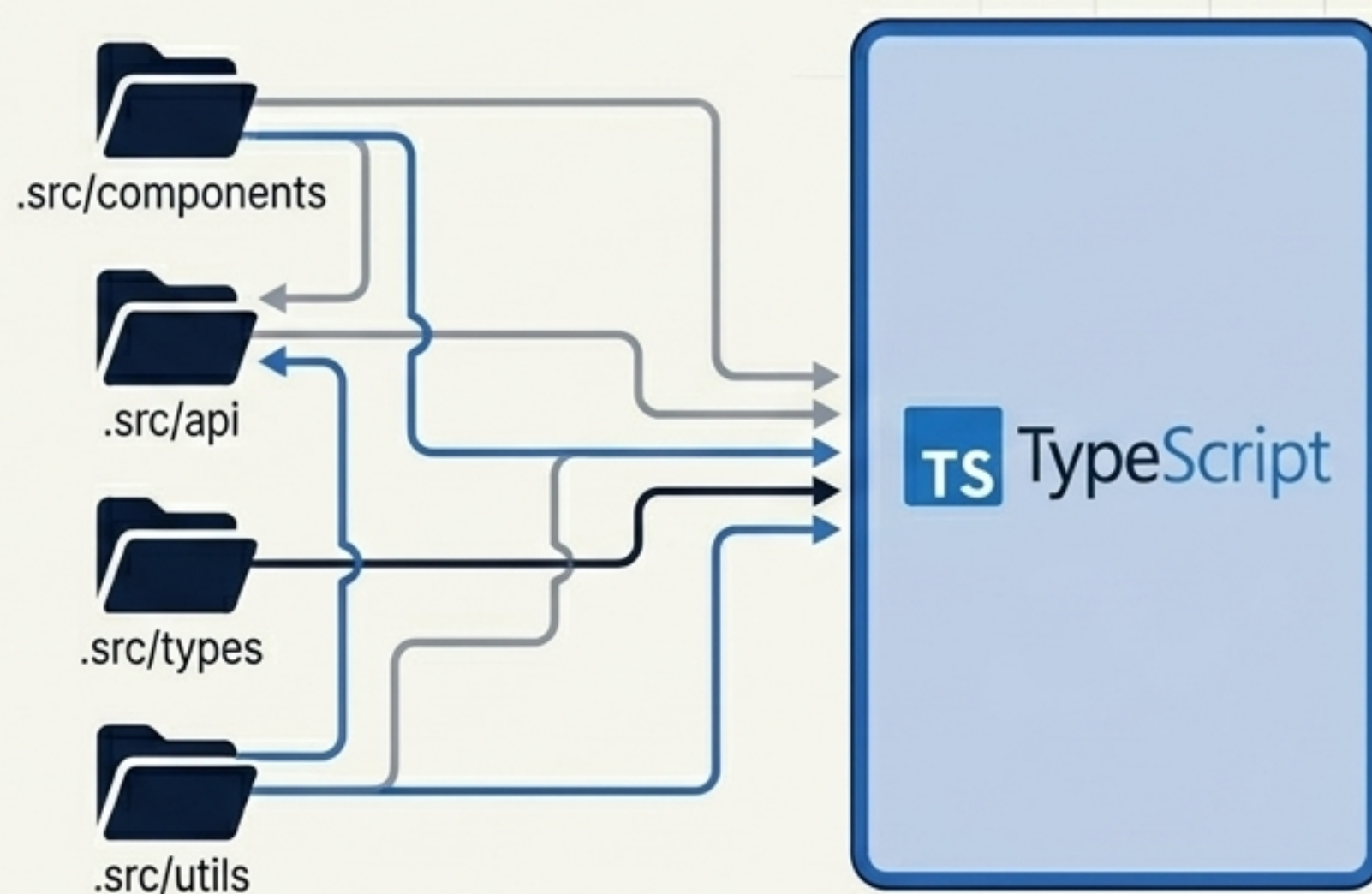
개발 패러다임 전환: 부분 수정(Micro)에서 전체 재설계(Macro)로

기존 방식 (Find & Replace 수준)



특정 파일의 버그 디버깅,
단일 함수의 로직 최적화

AX 기반 방식 (Architecture Migration)



수십 개 폴더 간의 종속성을 통째로
재배치하는 대규모 리팩토링

대규모 공사를 위한 매크로 프롬프트(Macro-Prompt) 구조 설계

거시적 리팩토링 성공의 핵심은 명확한 '작업 지시서(PRD)'의 제공



1. Current State (As-is)

현재 레거시 아키텍처의 한계와 문제점 명확히 서술

1
2

기존 폴더 구조는 컴포넌트와 비즈니스 로직이 강하게 결합되어 있어...



2. Target Architecture (To-be)

지향하는 시스템 구조와 적용 원칙

1
2

새 구조는 Clean Architecture 패턴을 따르며, 의존성을 주입하는 형태로...



3. Boundary (Constraints)

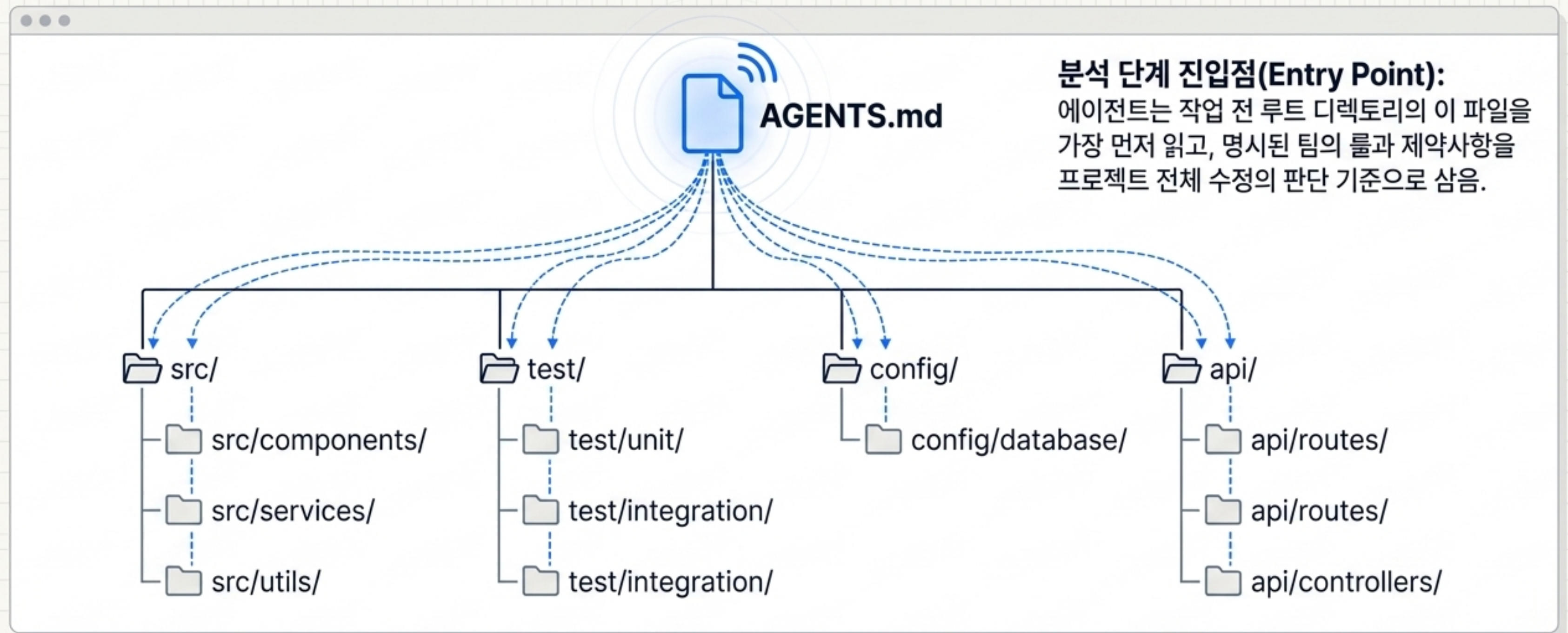
절대 수정해서는 안 되는 코어 비즈니스 로직 및 제약 조건 지정

1
2

결제 API 연동 모듈과 DB 마이그레이션 코드는 절대 건드리지 말 것...

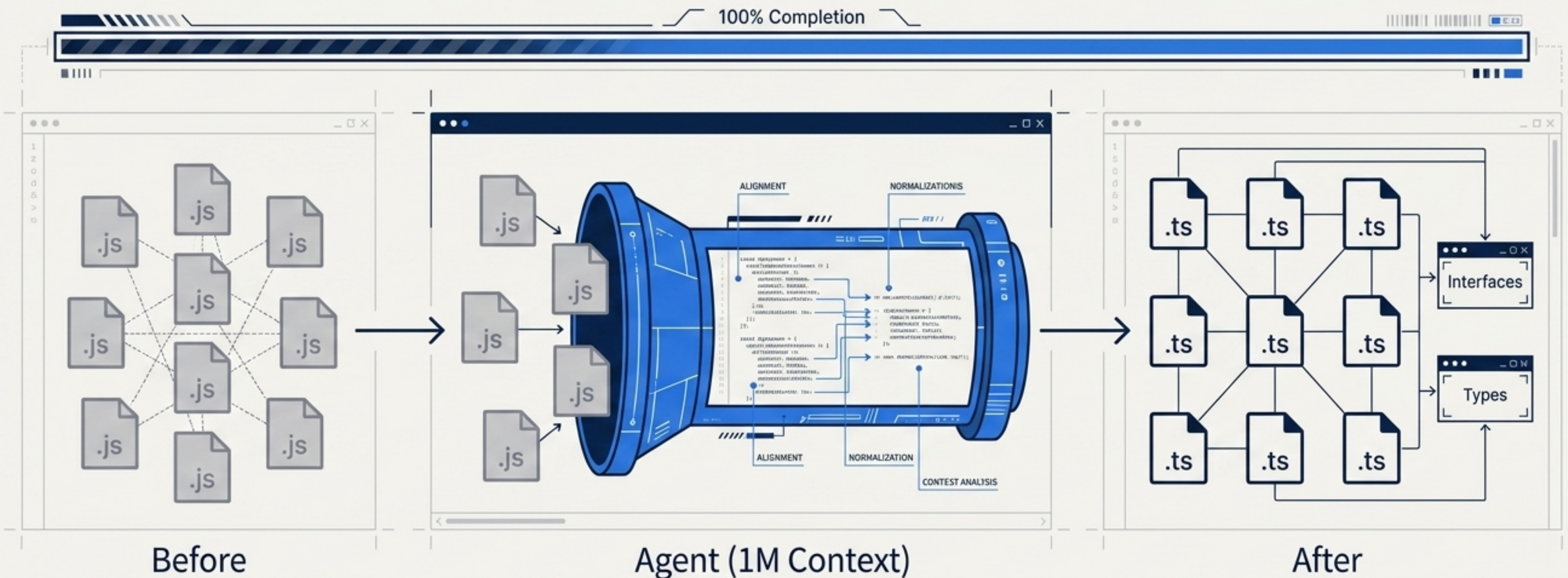
프로젝트 세계관 주입: AGENTS.md와 코딩 컨벤션 맵핑

100만 토큰의 거대한 컨텍스트 속에서 에이전트가 길을 잃지 않게 하는 등대 역할



Hands-on: 다중 파일 컨텍스트 마이그레이션 실습

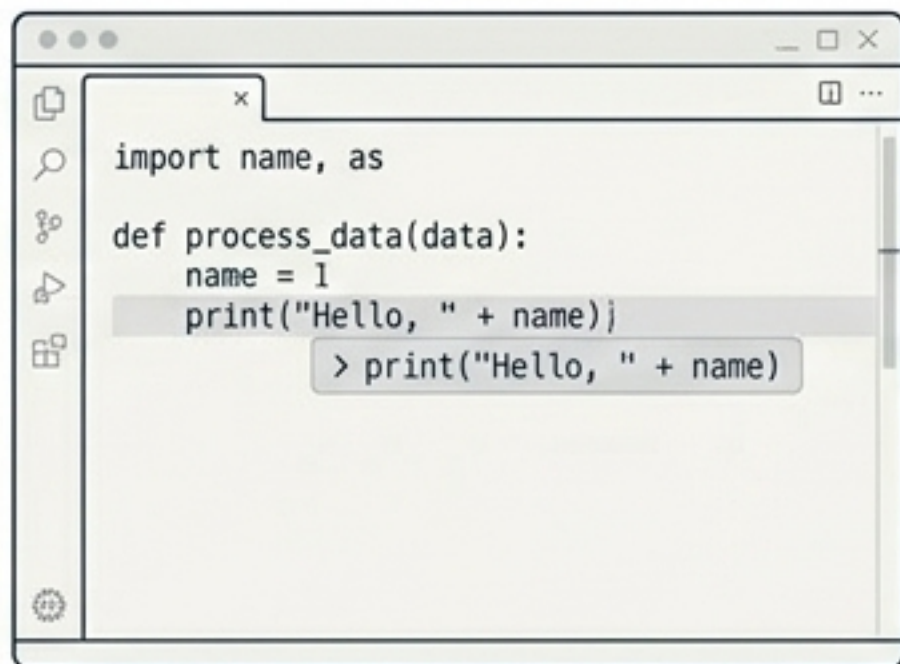
[실습 미션] 구형 JavaScript 모듈 10개를 TypeScript로 마이그레이션 및 인터페이스 정의 적용



단일 세션(Single Session)에서 에이전트가 10개 파일을 넘나들며 Type 일관성을 에러 없이 유지하는지 검증.

AX 시대를 이끄는 AI 코딩 도구 'Big 3' 해부

Toolscape



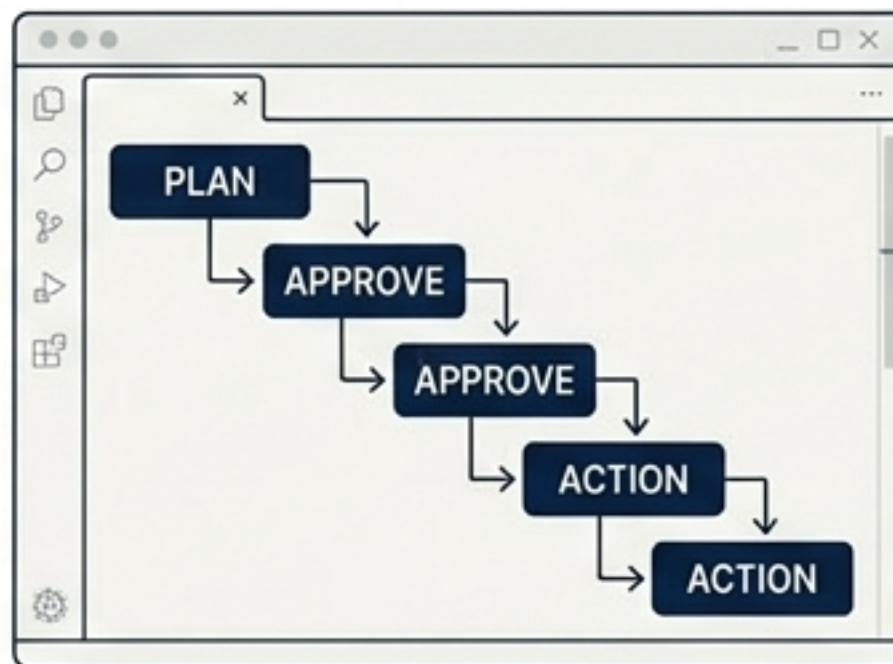
```
import name, as

def process_data(data):
    name = 1
    print("Hello, " + name)
    > print("Hello, " + name)
```

Cursor (IDE 내장)

기존 VS Code 환경 완벽 호환,
즉각적인 생산성 특화

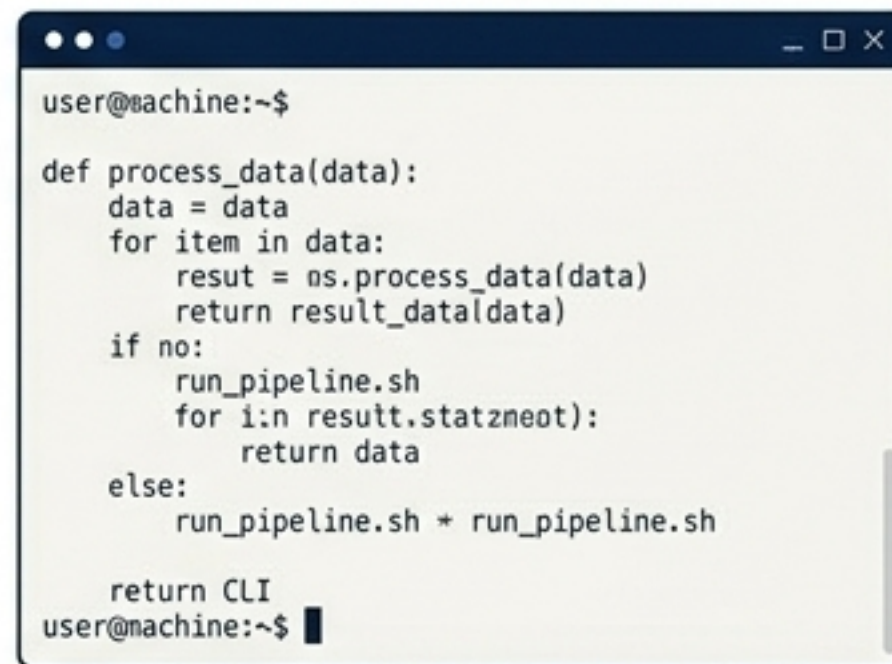
진입장벽 제로 / 빠른 수정



Windsurf (에이전트 IDE)

목표를 주면 계획(Plan)하고 승인 후
실행(Action)하는 직관적 워크플로우

목표 위임 / 계획 수립



```
user@machine:~$

def process_data(data):
    data = data
    for item in data:
        resut = ns.process_data(data)
        return result_data(data)
    if no:
        run_pipeline.sh
        for i:n result.statzmeent):
            return data
    else:
        run_pipeline.sh * run_pipeline.sh

    return CLI
user@machine:~$
```

OpenCode (터미널 에이전트)

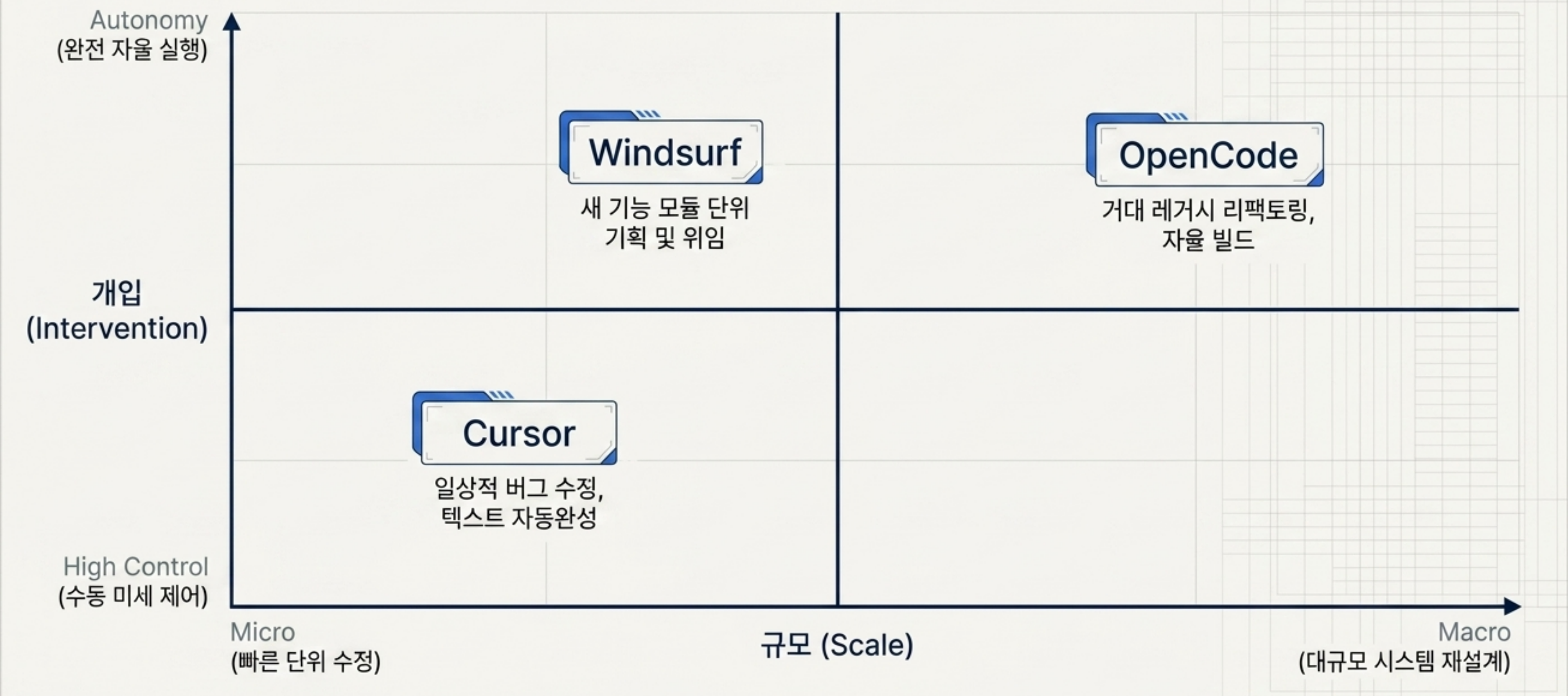
깊은 추론, 1M 컨텍스트 기반의 대규모
시스템 구축 및 자율 자동화 특화

터미널 네이티브 / 대규모 추론

도구별 핵심 역량 진단 테이블

구분	Cursor	Windsurf	OpenCode (Terminal)
핵심 강점	빠른 버그 수정, 인라인 자동완성	기능 단위 통째 위임, 직관적 UI	1M 컨텍스트, 자율 빌드
상대적 약점	⚠️ 복잡한 구조 추론	⚠️ 세밀한 마이크로 제어 한계	⚠️ 시각적 피드백 부족
실무 적합성	일상적 코딩, 단일 파일	신규 모듈 기획 및 개발	대규모 레거시 리팩토링

실무 목적별 AI 도구 선택 매트릭스 (Director's Matrix)



비즈니스 시나리오 매칭 워크샵

우리 팀의 과제에 가장 적합한 도구를 맵핑해 보십시오.

시나리오 A

기존 서비스에서 버튼 색상 변경 및 API 엔드포인트 URL 교체 (빠른 버그 수정)

시나리오 B

관리자 페이지의 새로운 '회원 통계' 메뉴 기획부터 구현까지 (기능 단위 위임)

시나리오 C

서비스 런칭 전, 하드코딩된 변수들을 전부 환경변수(.env)로 분리하고 배포 스크립트 작성 (전체 구조 변경)

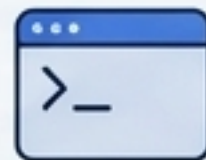
Toolbox



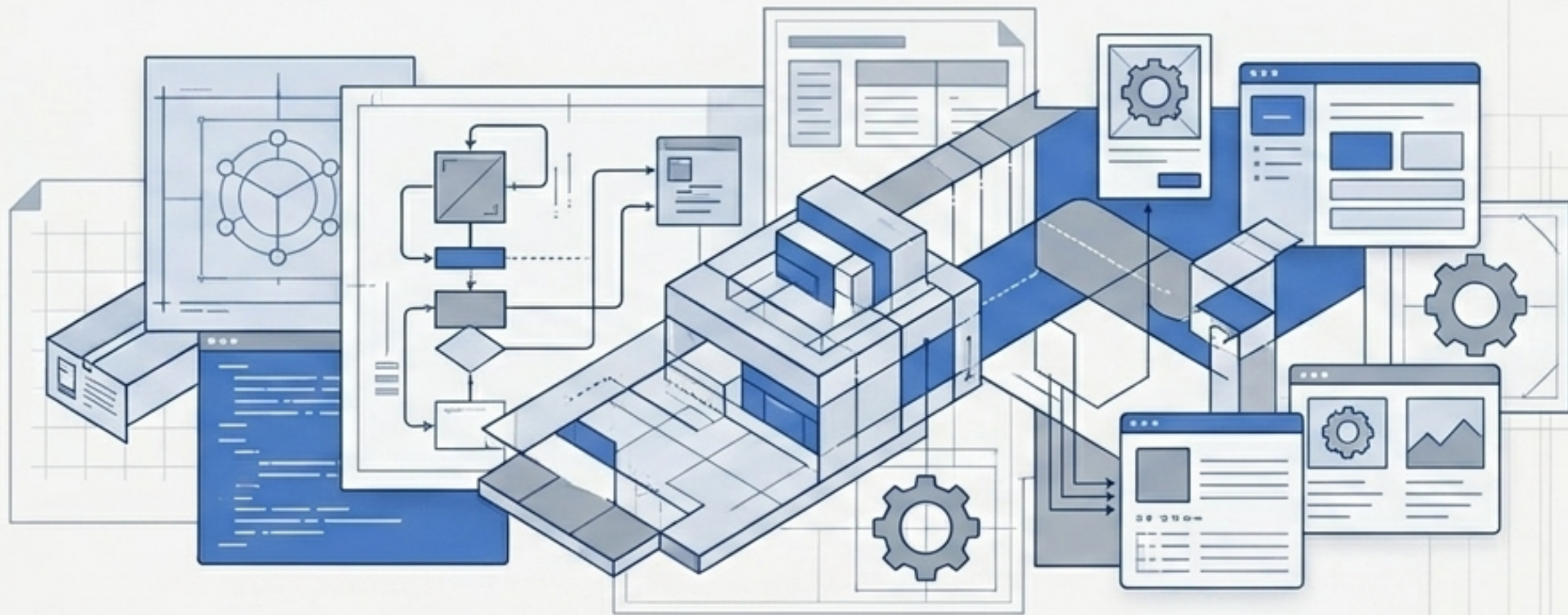
Cursor



Windsurf



OpenCode



“

도구가 중요한 것이 아니라,
워크플로우에 맞는 조합을 찾는 것이 핵심이다.

**AI-Native Engineer: 코드를 짜는 사람(Coder)이 아닌,
에이전트들을 조율하는 지휘자(The Conductor of Agents).**